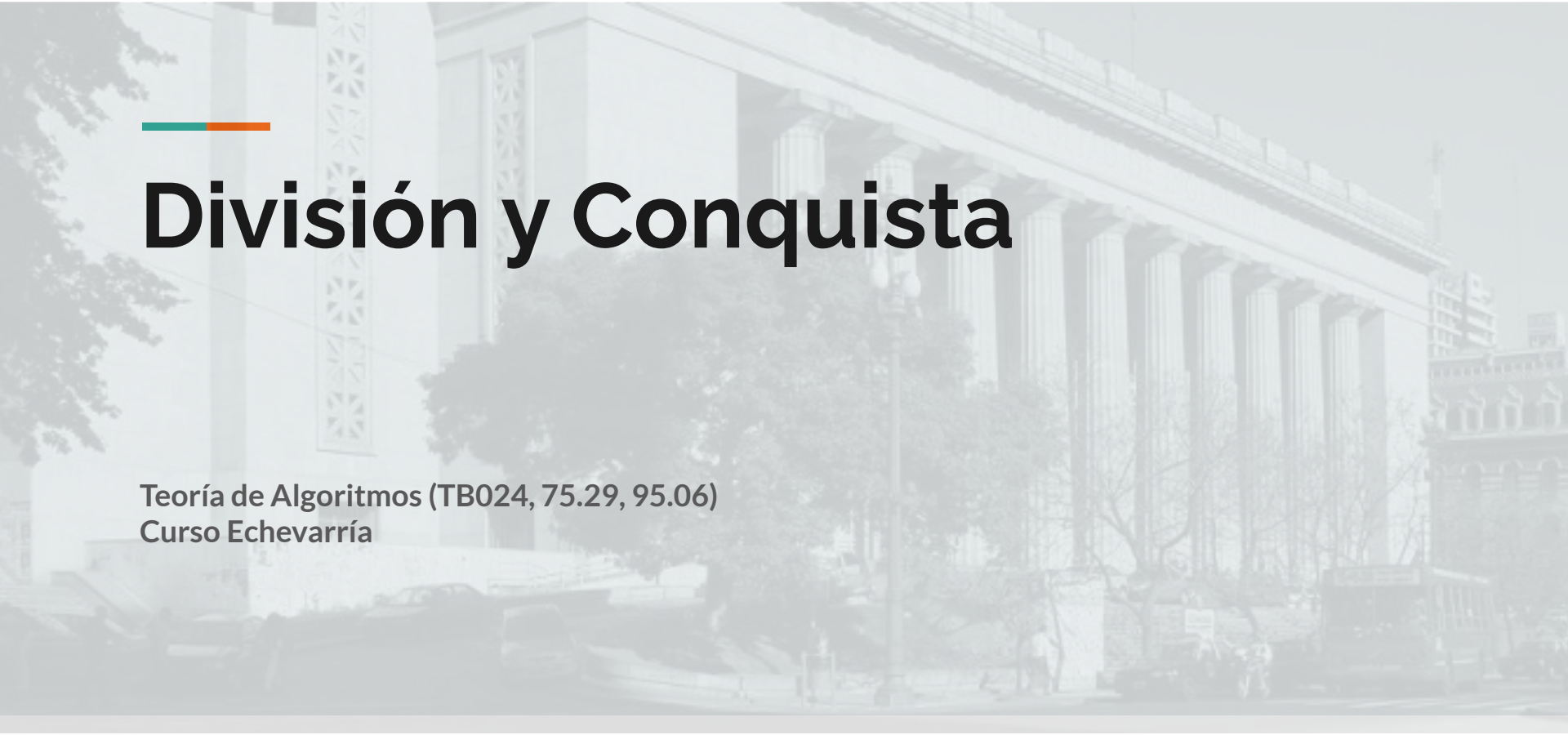
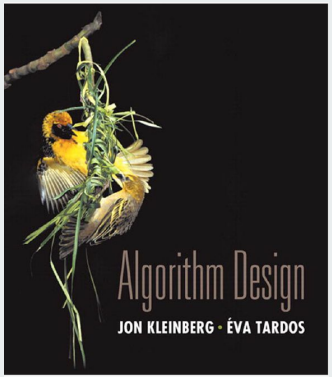


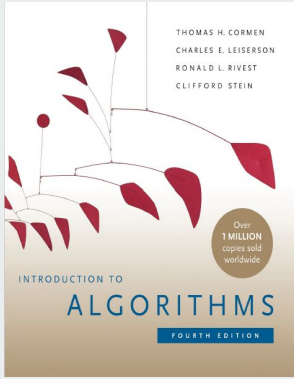
División y Conquista

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

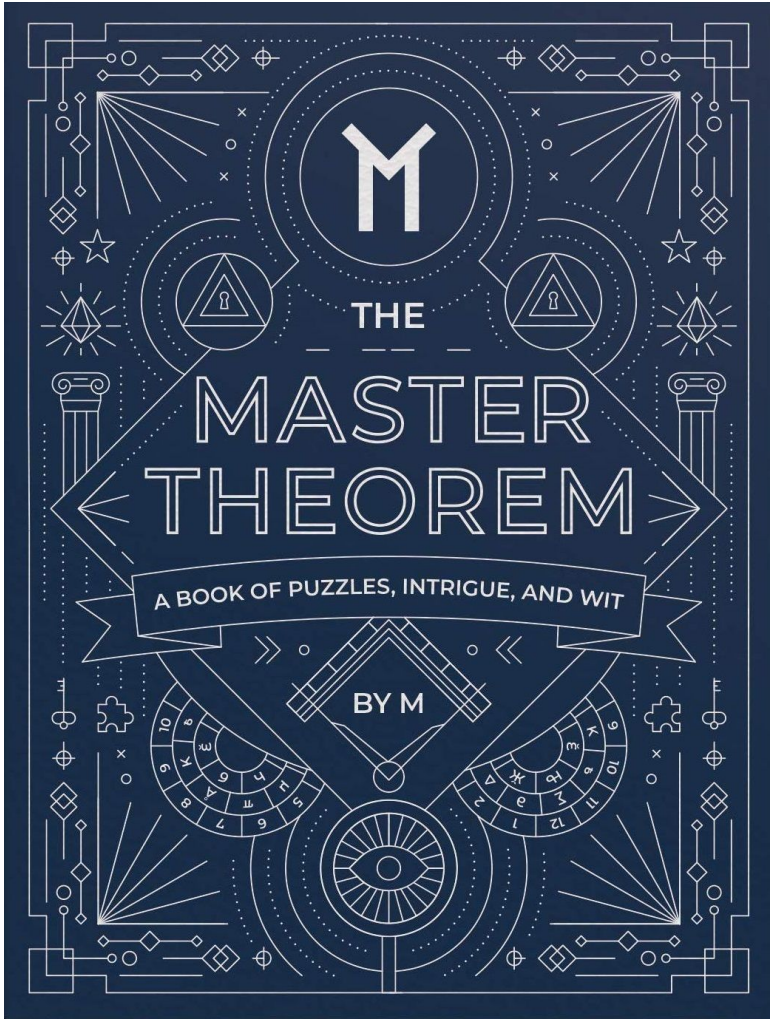
Teorema Maestro



Cap 5



Cap 4



Algoritmos recursivos

Supongamos que tenemos el siguiente pseudocódigo:

```
def p(X de tamaño n):
    si n < una constante k:
        resolver x y devolver el resultado
    si no:
        crear subproblemas de X, de tamaño n/b
        llamar recursivamente a p() sobre cada
    subproblema
        Combinar los resultados de los problemas
```

Complejidad Temporal del Algoritmo Recursivo

La ecuación de recurrencia del anterior algoritmo es:

$$T(n)=A\ T(n/B) + O(n^C)$$

Donde:

- A: Cantidad de llamadas recursivas
- B: Proporción de los subproblemas
- C: costo de combinar los sub-resultados

El Teorema Maestro

Dada una ecuación de recurrencia:

$$T(n) = a\,T(\frac{n}{b}) + O(n^c)$$

El Teorema Maestro indica como cota superior:

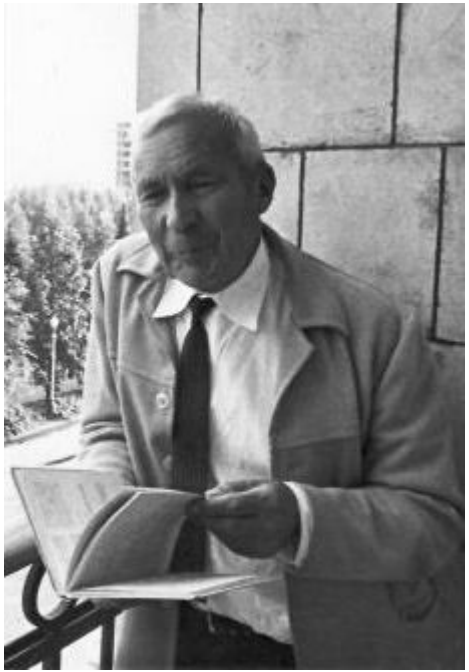
$$O(T(n))) = \begin{cases} O(n^{log_b\,a}) & : \; log_b\,a > c \\ O(n^{log_b\,a} log\,n) & : \; log_b\,a = c \\ O(n^c) & : \; log_b\,a < c \end{cases}$$

Algoritmo de Karatsuba

La anécdota de Kolmogorov y Karatsuba

- Corría 1952, y Andrey Kolmogorov, un académico ruso, intentó probar que la multiplicación de números enteros no podía ser mejor que $O(n^2)$
- Ya en 1960, durante un seminario de problemas matemáticos de complejidad computacional, presentó su postulado (aún sin prueba).
- Una semana después, Anatoly Karatsuba (de 23 años) encontraba un algoritmo capaz de multiplicar dos números de n dígitos en $O(n^{log_2 3})$, o bien, $O(n^{1,58})$
- Kolmogorov comenzó a presentar el algoritmo de Karatsuba e incluso lo publicó en un paper, citando al autor Karatsuba.
- Curiosamente, Karatsuba se enteró recién cuando el editor le envió las copias del paper.

Kolmogorov y Karatsuba



Andrey Nikolaevich Kolmogorov
1903-1987



Anatoly Alexeyevich Karatsuba
1937-2008

El producto de dos enteros *largos*

Supongamos que queremos multiplicar dos números x, y de n dígitos en la base B. Si $m<n$, podemos expresarlos como:

$$x = x_1 B^m + x_0$$

$$y = y_1 B^m + y_0$$

Entonces:

$$x\,y = (x_1 B^m + x_0)(y_1 B^m + y_0) =$$

$$= x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

El producto de dos enteros *largos* (cont.)

Notar que tenemos cuatro productos (x_1y_1, x_1y_0, x_0y_1, x_0y_0) que podemos resolver recursivamente.

La ecuación de recurrencia (suponiendo $m=n/2$) será:

$T(n)=4\,T(n/2) + O(n^1)$ -> $\log_2 4 = 2 > 1$

Y por el Teorema Maestro, nuestra multiplicación es:

$O(n^2)$

Parece que no refuta lo que decía Kolmogorov



El producto de dos enteros *largos* (cont.)

Karatsuba notó que se podía reemplazar esas cuatro multiplicaciones por tres multiplicaciones y un par de sumas adicionales

Reescribamos la multiplicación:

$$x\,y = (x_1\,B^m + x_0)(y_1\,B^m + y_0) = x_1\,y_1\,B^{2m} + (x_1 y_0 + x_0 y_1)B^m + x_0 y_0 =$$

$$= z_2\,B^{2m} + z_1\,B^m + z_0$$

- $z_2 = x_1 y_1$
- $z_1 = x_1 y_0 + x_0 y_1$
- $z_0 = x_0 y_0$

El producto de dos enteros *largos* (cont.)

Operando sobre z_1:

$$z_1 = x_1y_0 + x_0y_1 = x_1y_0 + x_0y_1 + (x_1y_1 - x_1y_1) + (x_0y_0 - x_0y_0) =$$

$$= x_1(y_1 + y_0) + x_0(y_1 + y_0) - x_1y_1 - x_0y_0 =$$

$$= (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0 =$$

$$= z_3 - z_2 - z_0$$

Ahora tenemos tres multiplicaciones y 4 sumas. Tiene que ser mejor que lo que teníamos antes.

El producto de dos enteros *largos* (cont.)

La nueva ecuación de recurrencia es:

$T(n)=3\,T(n/2)+O(n^1)$ -> $\log_2 3 > 1$

Y por el Teorema Maestro, nuestra multiplicación es:

$O(n^{\log_2 3}) \approx O(n^{1,58})$

El producto de dos enteros *largos* (cont.)

Pseudocódigo:

```
def multiplicacionBigInt(X, Y):
    si largo de X e Y son pequeños:
        return X*Y
    si no:
        X = x1 B^(n/2) + x0
        Y = y1 B^(n/2) + y0
        p = multiplicacionBigInt(x1 + x0, y1 + y0)
        x0y0 = multiplicacionBigInt(x0, y0)
        x1y1 = multiplicacionBigInt(x1, y1)
        return x1y1 B^n + (p - x1y1 - x0y0) B^(n/2) + x0y0
```

Conteo de Inversiones



Tima Miroshnichenko

Disclaimer

1. El Conteo de Inversiones no tiene nada que ver con inversiones financieras
2. No, no nos vamos a llenar de plata con esto

Definición del Problema

Supongamos que tenemos un conjunto de n elementos.

Alguien los ordena de una manera, y otra persona los ordena de una forma distinta

¿Se puede definir una métrica que diga qué tan “diferentes” son los dos ordenamientos?

Inversiones

Dos índices $i<j$ forman una *inversión* cuando $a_i > a_j$.

Si contamos la cantidad de inversiones entre dos listas, sabremos qué tan desordenadas están entre sí.

Inversiones (cont.)

Dos Listas:

El Señor de los Anillos	Duro de Matar
Duro de Matar	El Señor de los Anillos
Matrix	Matrix
Hombres de Negro	Titanic
Oppenheimer	Oppenheimer
John Wick	John Wick
El Libro Verde	Hombres de Negro
Titanic	El Libro Verde

Inversiones (cont.)

1. Etiquetamos los elementos de la primer lista de 1 a n
2. Ubicamos las etiquetas según el orden de la segunda lista de elementos

1 El Señor de los Anillos	2 Duro de Matar
2 Duro de Matar	1 El Señor de los Anillos
3 Matrix	3 Matrix
4 Hombres de Negro	8 Titanic
5 Oppenheimer	5 Oppenheimer
6 John Wick	6 John Wick
7 El Libro Verde	4 Hombres de Negro
8 Titanic	7 El Libro Verde

3. Identifiquemos las “inversiones”: (2;1), (8;5), (8;6), (8;4), (8;7), (5;4), (6;4) (son 7 en total)

Definición (cont.)

¿Y si la segunda lista estuviera ordenada exactamente al revés?

1 El Señor de los Anillos	8 Titanic
2 Duro de Matar	7 El Libro Verde
3 Matrix	6 John Wick
4 Hombres de Negro	5 Oppenheimer
5 Oppenheimer	4 Hombres de Negro
6 John Wick	3 Matrix
7 El Libro Verde	2 Duro de Matar
8 Titanic	1 El Señor de los Anillos

Inversiones: (8;7), (8;6), …, (2;1) (son 28 en total, o bien, $n(n-1)/2$)

En cambio, si ambas listas tuvieran el mismo ordenamiento, la cantidad de inversiones sería 0

Algoritmo (naive) para contar inversiones

La forma más simple de contar las inversiones es iterando para cada i y para cada j.

Eso es $O(n^2)$

¿Se puede hacer mejor?



Algoritmo (D&C) para contar inversiones

Podríamos adaptar Mergesort para contar inversiones

¿Qué pasaría si dividiera la lista en dos, y cada sublista ya estuviera ordenada?

¿Cuántas inversiones tendría en *cada* sublista?

¿Y *entre* las dos sublistas?

1 El Señor de los Anillos
2 Duro de Matar
3 Matrix
8 Titanic

4 Hombres de Negro
5 Oppenheimer
6 John Wick
7 La Lista Verde

Algoritmo (D&C) para contar inversiones (cont.)

[2138|5647] *(1,2,3,4,5,6,7,8) 4+1+2*

[21|38] *(1,2,3,8) 0+1+0* [56|47] *(4,5,6,7) 2+0+0*

[2|1] *(1,2) 1* [3|8] *(3,8) 0* [5|6] *(5,6) 0* [4|7] *(4,7) 0*

[2] [1] [3] [8] [5] [6] [4] [7]

Total: 7 inversiones

Algoritmo (D&C) para contar inversiones (cont.)

Relación de Recursividad:

$$T(n)=2\,T(n/2)+O(n^1) \quad \text{->} \quad \log_2 2 = 1$$

Por el Teorema Maestro:

$$T(n)=O(n \log n)$$

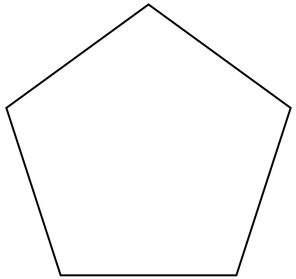
Punto extremo en Polígono Convexo

Definiciones

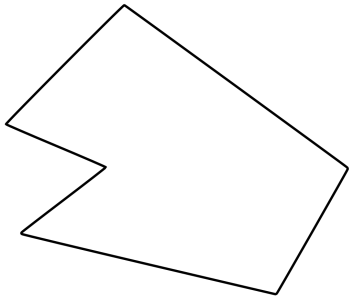
¿Cómo es un Polígono Convexo?

- Sus ángulos interiores son menores o iguales a 180º
- Es monotónico para todo segmento L (a lo sumo cruzará dos veces el polígono
- Todo par de puntos del polígono pueden ser unidos por un segmento totalmente incluido en él

Polígono Convexo



Polígono no Convexo



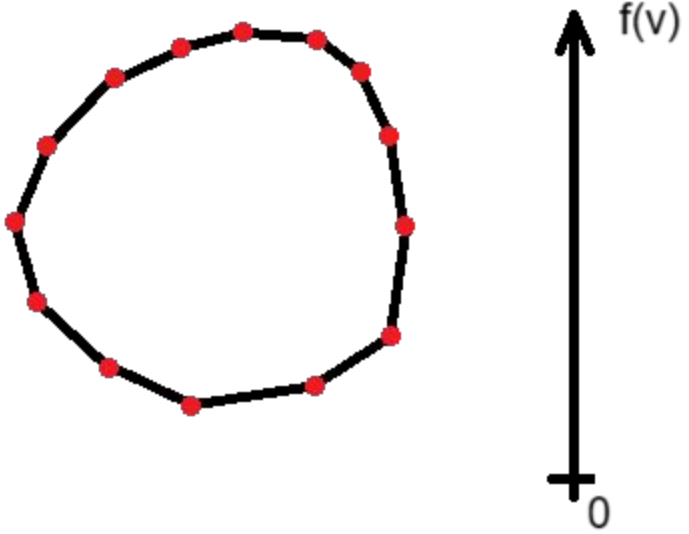
Definiciones (cont.)

Punto Extremo de una función

- Aquél punto x donde una función $f(x)$ toma el máximo o mínimo valor

Problema del Punto Extremo

Dado un conjunto de vértices $V=\{v_1, v_2, \ldots, v_n\}$, encontrar cuál de ellos es punto extremo de $f(v)$



Algoritmo (naive) para hallar Punto Extremo

Para cada v_i en V, evaluar $f(v_i)$ y quedarse con el que hace máxima (o mínima) a $f(v_i)$

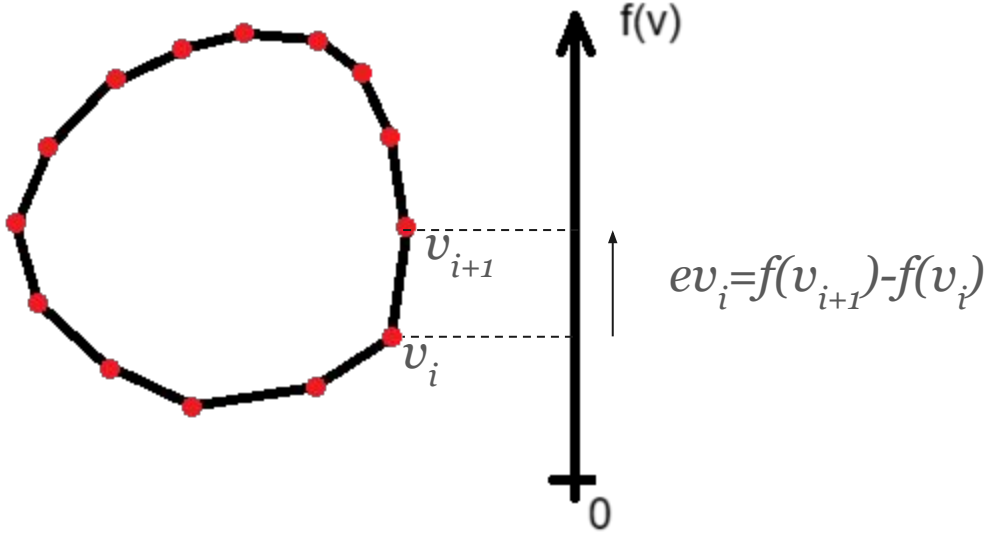
Complejidad: $O(n)$

No está mal, ¿pero se podrá hacer mejor?



Algoritmo (D&C) para hallar Punto Extremo

- Vamos a recorrer los vértices en sentido antihorario
- Definimos $ev_i = f(v_{i+1}) - f(v_i)$: es una medida del crecimiento de $f(v)$ (si nos acercamos/alejamos del punto extremo)



Algoritmo (D&C) para hallar Punto Extremo

- Dividimos el poliedro en dos, definiendo una secuencia de puntos entre v_a y v_b
- Tomamos un punto intermedio v_c de esa secuencia
- Analizamos ev_a y ev_c, y determinamos (criteriosamente) con qué tramo de la secuencia nos quedamos
- Cuando la secuencia tiene sólo tres elementos, comparamos $f(v_a)$, $f(v_b)$ y $f(v_b)$, y nos quedamos con el máximo/mínimo

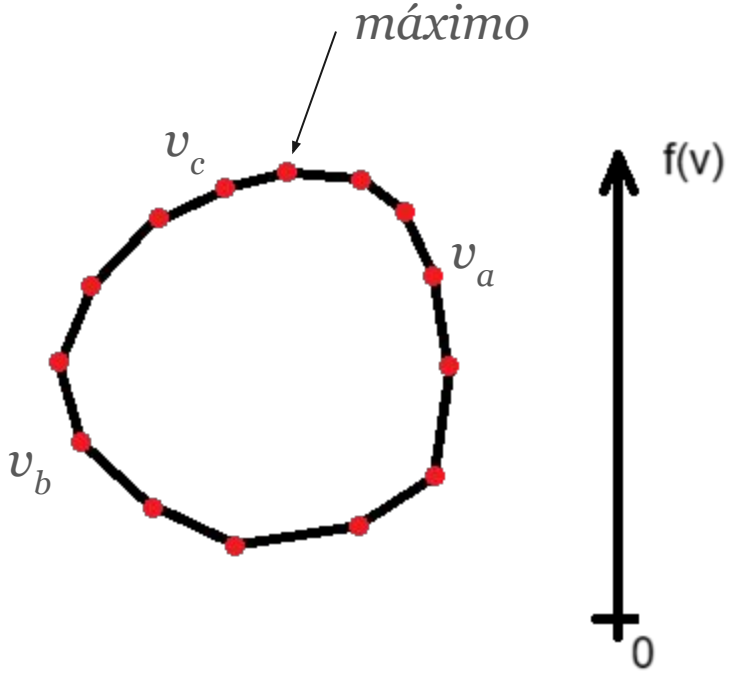
Algoritmo (D&C) para hallar Punto Extremo

Escenario 1:

- $ev_a > 0$
- $ev_c < 0$ (cambia el sentido)
- $f(v_c) > f(v_a)$

Me quedo con la secuencia:

$v_a \ldots v_c$



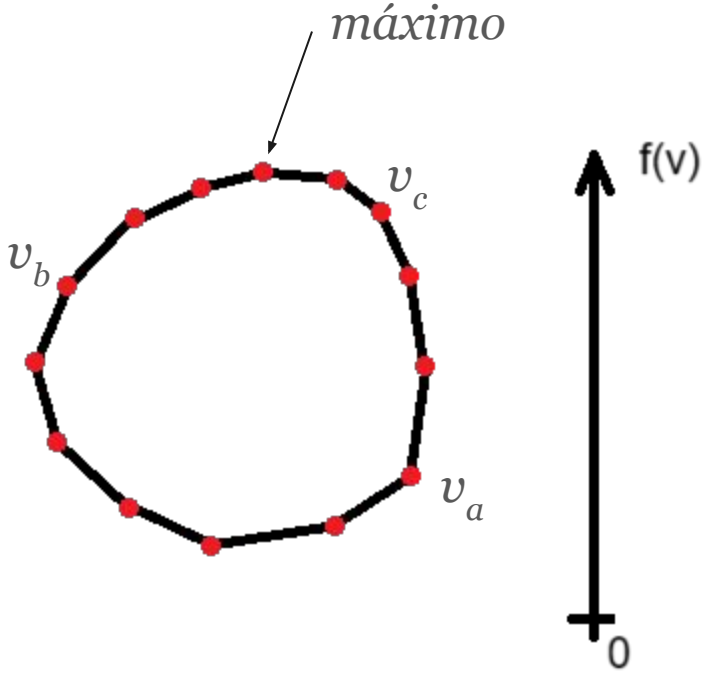
Algoritmo (D&C) para hallar Punto Extremo

Escenario 2:

- $ev_a > o$
- $ev_c > o$ (mantiene el sentido)
- $f(v_c) > f(v_a)$

Me quedo con la secuencia:

$v_c \ldots v_b$



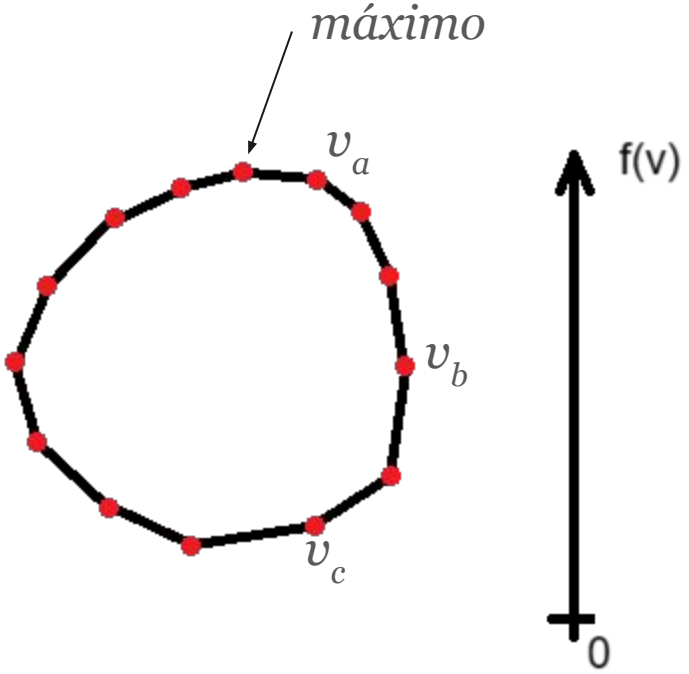
Algoritmo (D&C) para hallar Punto Extremo

Escenario 3:

- $ev_a > o$
- $ev_c > o$ (mantiene el sentido)
- $f(v_c) < f(v_a)$

Me quedo con la secuencia:

$v_a \ldots v_c$



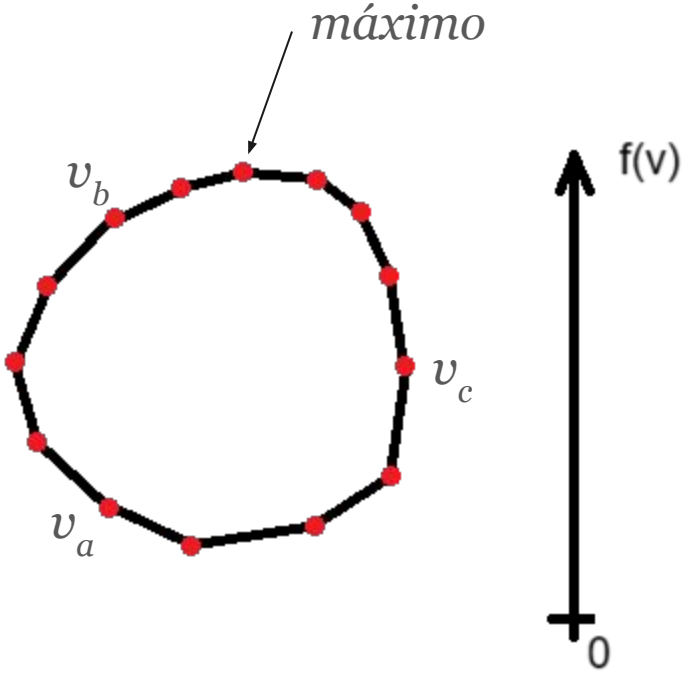
Algoritmo (D&C) para hallar Punto Extremo

Escenario 4:

- $ev_a < 0$
- $ev_c > 0$ (cambia el sentido)
- $f(v_c) > f(v_a)$

Me quedo con la secuencia:

$v_c \ldots v_b$



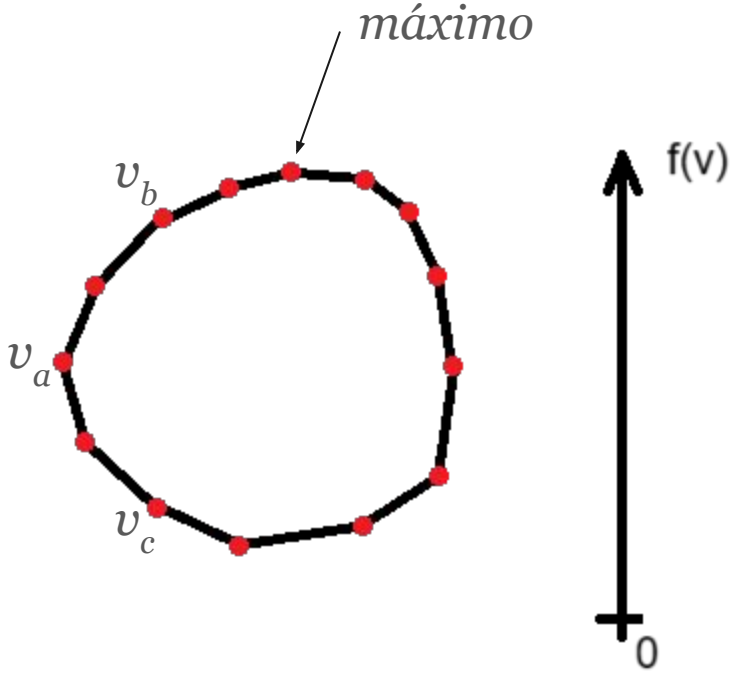
Algoritmo (D&C) para hallar Punto Extremo

Escenario 5:

- $ev_a < o$
- $ev_c < o$ (mantiene el sentido)
- $f(v_c) < f(v_a)$

Me quedo con la secuencia:

$v_c \ldots v_b$



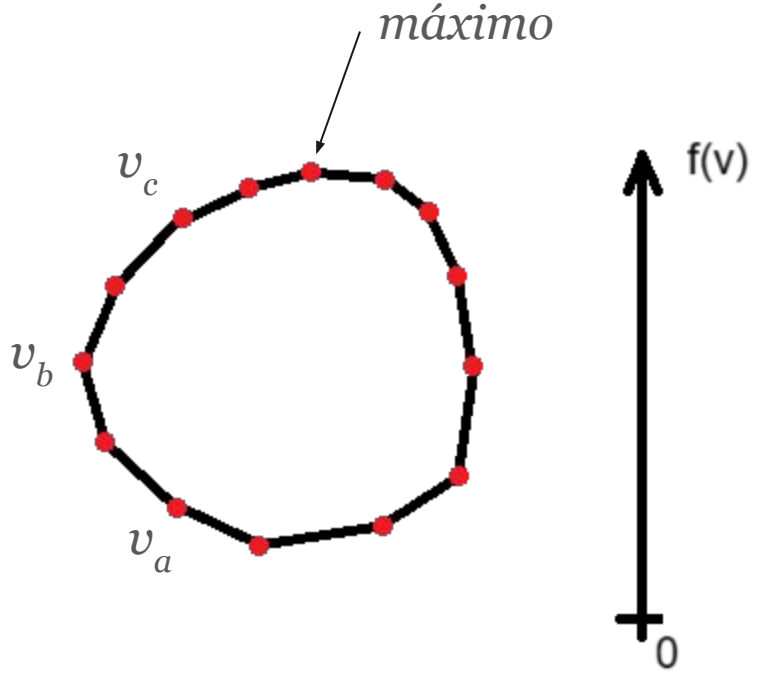
Algoritmo (D&C) para hallar Punto Extremo

Escenario 6:

- $ev_a < o$
- $ev_c < o$ (mantiene el sentido)
- $f(v_c) > f(v_a)$

Me quedo con la secuencia:

$v_a \ldots v_c$



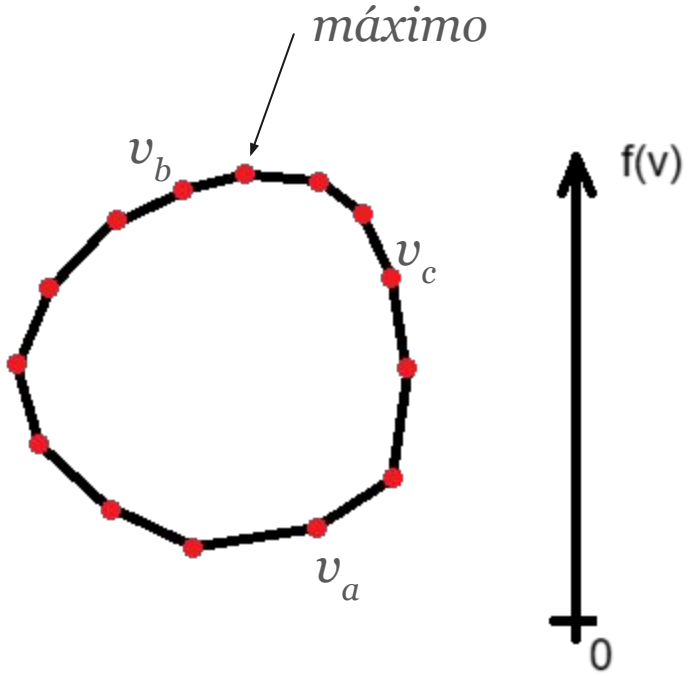
Algoritmo (D&C) para hallar Punto Extremo

Solución: Paso 1

- $ev_a > 0$
- $ev_c > 0$ (conserva el sentido)
- $f(v_c) > f(v_a)$ (me acerco)

Me quedo con la secuencia:

$v_c \ldots v_b$



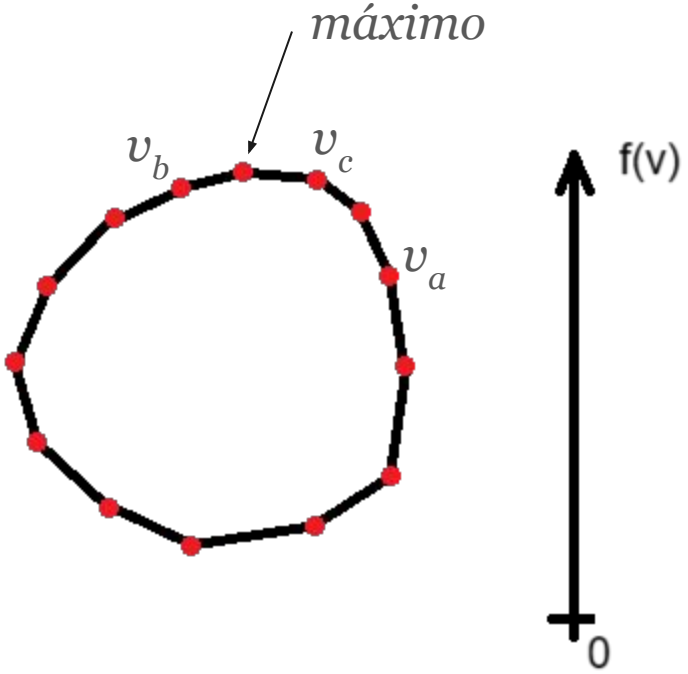
Algoritmo (D&C) para hallar Punto Extremo

Solución: Paso 2

- $ev_a > 0$
- $ev_c > 0$ (conserva el sentido)
- $f(v_c) > f(v_a)$ (me acerco)

Me quedo con la secuencia:

$v_c \ldots v_b$

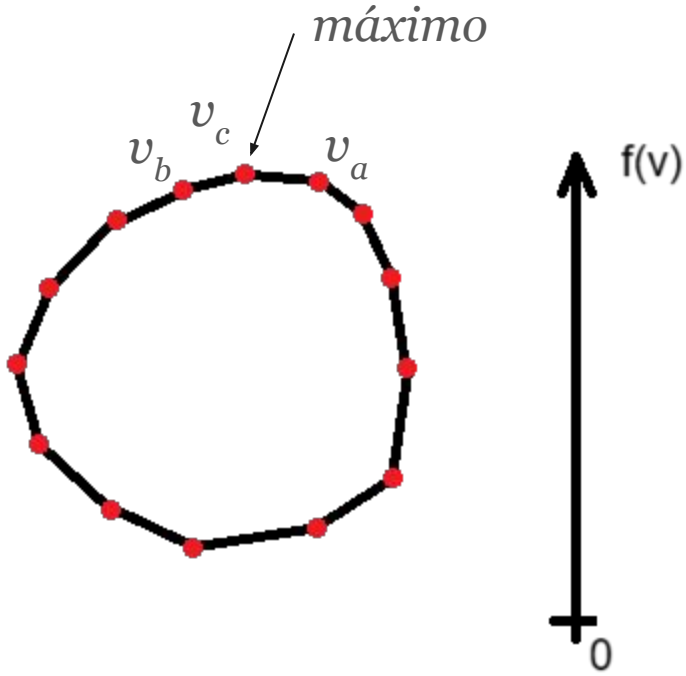


Algoritmo (D&C) para hallar Punto Extremo

Solución: Paso 3

Es el caso base

- $f(v_c)>f(v_a)>f(v_b)$: v_c es el máximo



Algoritmo (D&C) para hallar Punto Extremo

Resumen:

1. Iniciar con todos los puntos del polígono y determinar v_c (punto intermedio)
2. Si sólo quedan 3 vértices, compararlos y obtener el punto extremo
3. Si no, determinar qué escenario aplica
4. Actualizar v_a, v_b, v_c según el escenario y hacer la recursión

Algoritmo (D&C) para hallar Punto Extremo

¿Complejidad?

$$T(n) = 1\,T(n/2) + O(1) \quad \text{->} \quad \log_2 1 = 0$$

Por el Teorema Maestro:

$$T(n) = O(\log n)$$

Nota : O(1) = O(n^0)

Otros problemas de D&C

Otros problemas de D&C

- Puntos más cercanos en el plano
- Convoluciones y FFT (Transformada Rápida de Fourier)
- Mediana de dos conjuntos



¡Muchas gracias!